Machine Learning in Science
Assignment 2
Group 15 - Three Musketeers:

Vlad Chibulcutean (1780980)
Andreas Sinharoy (1804987)
Alex Gavriliu (1785060)

Pt1. Problem Analysis:

A.

Dimensionality:

Dimensionality refers to the number of independent variables or features in a dataset, representing the complexity and breadth of the data's structure. The problem of the polygonal face has one set of inputs and one set of outputs we need to use to solve the problem. Our inputs here are the 7 lengths ($l_1, l_2, ..., l_7$) of the theoretical polygon, and our output is the maximum possible area of the polygonal face these lengths form ($A_{max}(l_1, l_2, ..., l_7)$). The unit of measurements (yardstick) here is unspecified, however we know the input is a length measuring some distance (like metres, centimetres) and the output is an area measuring the size of some 2-dimensional region (m^2, cm^2). We also assume that the unit (yardstick) of the areas here is the unit of the length squared.

Dimensional Homogeneity:

Secondly, dimensional homogeneity ensures that all terms in an equation possess the same dimensional units, thereby guaranteeing physical coherence and interpretative validity. To make our problem dimensionally homogeneous, there are a number of different strategies that can be undertaken.

One straightforward method is simply squaring each of the inputs to make them have the same unit as the output. That is, given inputs $\{l_1, l_2, ..., l_7\}$ we change these to $\{l_1^2, l_2^2, ..., l_7^2\}$. This method ensures direct compatibility between the dimensions of inputs and outputs. However, it may destroy the relative magnitudes of the inputs, potentially affecting the model's sensitivity to small changes in individual variables.

Another method would involve square rooting the outputs (or areas),

again making the units of inputs and outputs equal. In this method, for an output area o, the new output would be $\sqrt{o}$. This approach maintains the original scale of the outputs while ensuring dimensional homogeneity. However, it may complicate interpreting the result, especially if it changes the data's distribution.

A third method could involve using the formula for the area of a square (πr^2), assuming that each of our inputs corresponds to the radius r of that square. Thus, for inputs $\{l_1, l_2, ..., l_7\}$ the new, dimensionally-homogeneous inputs would be $\{\pi l_1^2, \pi l_2^2, ..., \pi l_7^2\}$. This method simplifies the interpretation of the inputs as lengths or radii, making them directly comparable to the output area. However, it imposes a specific geometric interpretation on the data, which may not be appropriate for our problem.

Another method for homogenization could involve multiplying the different inputs among one another to create new dimensionally homogeneous inputs. One manner of doing this would be turning the inputs $\{l_1, l_2, ..., l_7\}$ into a new set of inputs $\{l_1 l_2, l_2 l_3, ..., l_6 l_7, l_7 l_1\}$. One concrete explanation of such a method would be to perform the cartesian product of a set of 7 edges with itself. Doing this means that from 7 lengths $\{l_1, l_2, ..., l_7\}$ you instead generate 49 inputs $\{l_{11}, l_{12}, l_{13}, ..., l_{75}, l_{76}, l_{77}\}$ that can be fed into the neural network. Of these 49 there would always at most be 28 unique inputs as there would be duplicate multiplications of edges, for example $l_{13} = l_{31}$ as these are just the same 2 edges being multiplied twice. This could potentially be filtered as to not unnecessarily feed useless data. Consequently, each of the inputs would have the same unit as the outputs, making the problem dimensionally homogeneous. An advantage of this method could be the capturing of interactions or relationships between the different inputs. Having a large number of raw inputs gives the computer a lot more relationships between single outputs and inputs so it will be able to make more informed and potentially accurate predictions. On the other hand, this could cause multicollinearity, which is characterised by features being excessively correlated. Consequently, it could be hard for the model to understand relationships between variables, making predictions less reliable and clear.

Finally, a last method to not only homogenise the data, but also scale it simultaneously, would be to divide all of the side lengths representing a given input polygon by the perimeter (sum of the side lengths):

$\frac{l_n}{\sum_{i=1}^{7} l_i}$: $l_n, l_i \in \{l_1, l_2, l_3, l_4, l_5, l_6, l_7\}$, and all maximal area output values by the perimeter squared: $\frac{A_{max}(l_1,l_2,l_3,l_4,l_5,l_6,l_7)}{(\sum_{i=1}^{7} l_i)^2}$. That is, for each set of inputs, each side length would be divided by the perimeter of the polygon represented by that set of side lengths, whilst the output maximal area would be divided by the perimeter squared. This homogenises the data by making the inputs and outputs unitless. These no longer measure a specific length (input) or area (output), instead what they are measuring is the ratio between the polygon's dimensions. The inputs are now the relative length of a side compared to the perimeter, and the outputs are now the area of the polygon relative to its perimeter squared. The second major benefit of this method is that it scales all data to $0 < X < 1$. This will also make it easier for the model to be trained as the jump between values is much smaller and will lead to greater stability in training. There aren't any real downsides to this method apart from the inherent difficulty of returning all the predictions to the correct scale.

We initially went with the third method of homogenization for our model as we already thought it was sufficient to attain an accurate enough model with the correct training parameters. However we found out after 2 days worth of testing that this method was not effective enough to drop our MAPE test error for our result below 0.6%. So in the end we implemented the final homogenization method combined with perimeter scaling/homogenization such that both inputs and outputs were unitless. It was able to much more easily reach a much lower error. It was able to reach lower than 0.1% with ease given the correct parameters for training, so it was the way to go for training our model.

Scaling:

Scaling in data processing is a critical step that involves adjusting the range or distribution of data values to match their magnitudes closer together, thus ensuring consistency across the dataset. By scaling the data, various algorithms can operate more effectively, as they become less sensitive to the different scales of the input features. This process ultimately enhances algorithm performance and facilitates more accurate and meaningful comparisons between different datasets or features. Several methods are commonly used to scale data, each with its own advantages and applications. These methods include

normalisation, which rescales the data to a standard range typically between 0 and 1 or -1 and 1, and standardisation, which transforms the data to have a mean of 0 and a standard deviation of 1, making it suitable for algorithms that assume a Gaussian distribution. Below are some scaling methods.

1. **Min-Max Method:**

Rescales the data to a fixed range, typically [0, 1].
Formula: $X' = (X - X_{min})/(X_{max} - X_{min})$

Min-max scaling is useful when the data needs to be bounded within a specific range, such as [0, 1]. It preserves the relationships between values while resizing them, ensuring that the relative differences remain consistent. This method is ideal for algorithms that require normalised inputs, such as neural networks, as it standardises the feature values within a specified range, facilitating effective model training.

2. **Standardisation Method:**

Rescales the data to have a mean of zero and a standard deviation of one.

Formula: $X' = (X - \mu)/\sigma$
Advantages:

Standardisation centres the data around zero, which is beneficial for algorithms that assume a Gaussian distribution, such as linear regression and logistic regression. It handles outliers better than min-max scaling by taking the data's distribution into account, making it a more robust method for datasets with varying scales and outlier presence.

3. **Robust Scaling Method:**

Rescales the data using the median and the interquartile range (IQR), making it robust to outliers.

Formula: $X' = (X - median)/IQR$

Advantages:

Robust scaling centres the data around the median and scales it based on the IQR, which makes it particularly effective for datasets with outliers. By using the median and IQR, robust scaling reduces the influence of extreme values, allowing it to handle data with significant outliers more effectively than standardisation or min-max scaling.

4. **Perimeter Normalisation:**

Rescales the data to be to a fixed range [0,1), using the perimeter of the polygon as the divisor (sum of edges).

Formula:

Inputs: $\frac{l_n}{\sum_{i=1}^{7} l_i}$: $l_n, l_i \in \{l_1, l_2, l_3, l_4, l_5, l_6, l_7\}$

Outputs: $\frac{A_{max}(l_1,l_2,l_3,l_4,l_5,l_6,l_7)}{(\sum_{i=1}^{7} l_i)^2}$

Advantages:

This scaling method not only has a scaling effect but also a homogenization effect which is why it is very strong and useful for nicely formatting the data before training. Not only does it cause all values to be in the interval [0,1), (does not include 1 as perimeter will always be larger than individual sides and perimeter squared will always be larger than area) But now all the data across the inputs and outputs are unitless and are instead raw ratios based on the dimensions of the polygon.

We tried to implement each of these methods in our code, but the model would underperform when scaled with either of these methods compared to when the data was unscaled. Consequently, we used the above mentioned method where we made the inputs and outputs unitless and simultaneously scaled.

B.

Acquired Equivariance Property and Equivariance Group

The equivariance property acquired by rendering the process dimensionally homogeneous is $f(\lambda x) = \lambda f(x)$ where λ is a constant/scalar, f is the function created by the neural network, and x is an input (in our case a list of

28 scaled inputs created by taking the cartesian product as explained above). Firstly, this is an equivariance property since its structure corresponds to that of $g \circ f = f \circ g$, where g represents the (scalar multiplication) transformation and f again represents the function created by our neural network. This correspondence holds because regardless of whether the transformation is done before or after the function is run on the input, the results remain the same.

The reason why this holds true within our model is based on the assumption that our model will only contain linear layers and linear activation layers. Since both of these layers are degree-1 homogeneous, and our neural network combines them, the neural network itself will ultimately end up being degree-1 homogeneous. Consequently, the above equivariance property, equivalent to degree-1 homogeneity, will also hold. Finally, our equivariance group consists of the above property of scalar multiplication.

Invariance Group Description on Basis of Cyclic Polygonality:

When Considering the invariance properties of the input data we must consider what this data means geometrically. When enclosing the maximal possible area of a set of sides the polygon created as a result of this is called a cyclical polygon. There are three trivial geometrical invariances that don't affect the area of the polygon when transformed. These are:

- **Reflection** of the enclosed shape in any line (axis) of the 2d plane it is constructed ()
- **Rotation** of the enclosed shape in on any axis of the 2d plane it is constructed ()
- **Translation** of the enclosed shape on the 2d plane it is constructed ()

Let us consider how these are represented in the aforementioned notation. If we have sides $l_1, l_2, l_3, l_4, l_5, l_6, l_7$ in this specified order we have that our max area is $A_{max}(l_1, l_2, l_3, l_4, l_5, l_6, l_7)$.

A **Rotation** of these sides would still just be represented by $l_1, l_2, l_3, l_4, l_5, l_6, l_7$ or some shift up or down of the values e.g. $l_3, l_4, l_5, l_6, l_7, l_1, l_2$. This is because the order of sides and their connections is preserved and only their position in space is changed. Thus $A_{max}(l_1, l_2, l_3, l_4, l_5, l_6, l_7) = A_{max}(l_3, l_4, l_5, l_6, l_7, l_1, l_2)$.

A **Reflection** of these sides would be represented by a reversal of the order and potential shift up or down of these sides. So a reflection could be represented by $l_7, l_6, l_5, l_4, l_3, l_2, l_1$ or $l_5, l_4, l_3, l_2, l_1, l_7, l_6$. Even though the order is reversed all

previous connections between edges are preserved and much like a rotation only their positions in space have changed, so area is unaffected. Thus $A_{max}(l_{1,}l_2, l_3, l_4, l_5, l_6, l_7) = A_{max}(l_7, l_6, l_5, l_4, l_3, l_2, l_1) = A_{max}(l_5, l_4, l_3, l_2, l_1, l_7, l_6)$.
A **Translation** of these sides would still be represented by $l_{1,}l_2, l_3, l_4, l_5, l_6, l_7$.

The side's rotation or order have not changed only their position in space, so it is clear that the area has not been changed.
The final invariance of this set of inputs however not only covers the previous trivial cases, but all other possible permutations of the edges $l_{1,}l_2, l_3, l_4, l_5, l_6, l_7$.

Consider the Cyclic polygon depicted in figure 1:

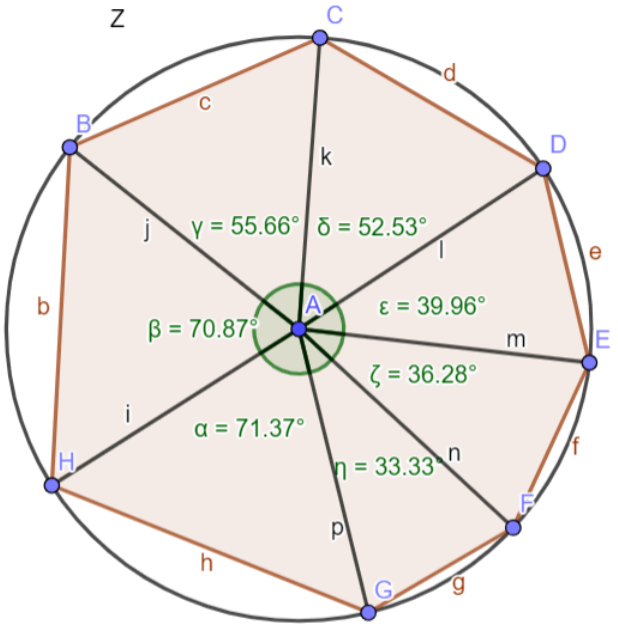


Figure 1 - Cyclic Polygon with points situated on circle Z

Above we have a cyclic polygon composed of 7 sides a, b, c, d, e, f, g; these points lie on circle Z (Z is the circumcircle of the polygon). This polygon is divided into 7 isosceles triangles by drawing 7 edges (i, j, k, l, m, n, p) equal to Z's radius from the centre of Z to the corners of the polygon. The angles that for at the centre of the circle from each triangle are denoted as $\alpha, \beta, \gamma, \delta, \varepsilon, \zeta, \eta$. We know definitively that $\alpha + \beta + \gamma + \delta + \varepsilon + \zeta + \eta = 360°$, as the sum of central angles of all sectors of a circle is 360°. What this tells us is that if we take these triangles and arbitrarily switch their position, such that the angle between the 2 equal sides is in the centre, they would fit perfectly as those angles always sum up to 360°. Furthermore, each isosceles triangle's pair of equal edges are the length of the radius and are thus also globally equal in

length. This means that any possible arrangement of the triangles, such that the angle between its 2 equal sides is in the centre, to form a polygon from the centre point of Z will have its vertices placed on the Z as it is the same distance away from the centre (radius). The area of each triangle that form these cyclical polygons is always the following 7 terms: $l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7$ $\frac{ip(sin(\alpha))}{2}, \frac{ij(sin(\beta))}{2}, \frac{jk(sin(\gamma))}{2}, \frac{kl(sin(\delta))}{2}, \frac{lm(sin(\varepsilon))}{2}, \frac{mn(sin(\zeta))}{2}, \frac{np(sin(\eta))}{2}$. This means that the area of all these cyclical polygons will always be:
$\frac{ip(sin(\alpha))}{2} + \frac{ij(sin(\beta))}{2} + \frac{jk(sin(\gamma))}{2} + \frac{kl(sin(\delta))}{2} + \frac{lm(sin(\varepsilon))}{2} + \frac{mn(sin(\zeta))}{2} + \frac{np(sin(\eta))}{2}$
What this demonstrates mathematically, is that no matter how the edges a, b, c, d, e, f, g are connected to form a maximal cyclic polygon, the area of these polygons is **always the same**. So, in the context of edges $l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7$, this means that:
where $S(l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7)$ denotes the set of all possible permutations of the set of edges $\{l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7\}$,
$A_{max}(l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7) \;= s:\, s \in S(l_{1,}\, l_2, l_3, l_4, l_5, l_6, l_7)$.

C.

Methods for Respecting Problem Symmetry

To have our model respect our problem's symmetry, we can employ three distinct methods discussed in the lectures.

1. **Augmentation:**

The first such method is augmentation, which involves expanding the dataset by (essentially) multiplying it by the symmetries. Since the lengths in the input can be ordered in any way without impacting the maximum area, it follows that for any given one input $i = \{l_1, l_2, ..., l_7\}$, the dataset can be augmented with all of the different permutations of i. The range of these permutations would account for all of the different symmetries of the problem, which were discussed above. One input consists of 7 digits, so it follows that there will be $7! = 5040$ different inputs for every pre-existing input, increasing the total number greatly. We must also note that for all of these different permutations, the maximum area corresponding to the inputs (i.e., the correct

output) would be the same. In this way, the problem symmetry would be respected since the model would learn to consider all of the different permutations of a given input as having the same output value. However, augmentation is not an exact method, rather only approximating the symmetry. Moreover, due to the significantly enhanced number of inputs, the model could take longer to train and its computational load could increase, in addition to the existing risk for overfitting. All in all, due to its notable limitations, we decided against using this method.

2. **Disambiguation:**

A second method described in the lectures is disambiguation, which involves applying symmetries to the data before feeding it to the neural network, essentially combining all symmetric inputs into one. Consequently, the size of the inputs can be decreased considerably, leading to lower training time and computational load. For our specific problem, the manner in which disambiguation could be carried out is by ordering all of the inputs from smaller to bigger side lengths, and then removing any duplicates. This will account for all of the problem's symmetries, since they can all be represented by permutations of the inputs, and all permutations would be turned into the same input by this method. Thus, the model will only be fed data in this one specific format, and it will learn to consider symmetrical inputs as identical. This method is also defined in the lectures as exact, creating a perfect combination between ease of use and effectiveness.

3. **Hardwiring:**

Lastly, the third method discussed in the lectures is hardwiring, which involves "hardwiring" a neural network to respect a problem's symmetries. This method offers efficiency, reduced training data requirements, and improved performance in favourable scenarios. Moreover, it is generally perceived to have predictable and transparent behaviour, which helps in ensuring consistency. On the other hand, hardwiring implementations can lack flexibility and scalability, struggle with new or unseen data, and risk overfitting. Furthermore, its development generally requires significant effort and expertise, as does maintaining and updating its rules. All in all, for our problem's scope, its disadvantages seem to outweigh the advantages, rendering hardwiring too complicated a strategy.

Ultimately, we decided to use disambiguation to adapt our dataset to respect the problem's symmetry using the ordering method described under the disambiguation explanation above. Though this may not end up being as elegant or integrated a solution as hardwiring could have been, its exactness and other advantages make it an undoubtedly great solution.

Description of a Simple Neural Network

Our neural network architecture can be designed to be both simple and effective. The architecture comprises several key components, each serving a specific purpose, and each of which we will quickly discuss.

The input layer consists of 28 neurons, matching the cartesian product of the number of side lengths of the enclosed fence. Each neuron in the input layer is fully connected to every neuron in the next layer by multipliers, facilitating the propagation of information.

Hidden layers play a crucial role in capturing the complexity of the relationship between the side lengths and the maximum enclosed area. One to five hidden layers can be employed, with the number of neurons in each layer being chosen experimentally. In PyTorch, these hidden layers are implemented as linear layers. The number of neurons per hidden layer will either stay constant across the network, or decrease with each new hidden layer. Moreover, the number of neurons per hidden layer will be experimented with to find the ideal solution.

Activation functions are applied to the output of hidden layers to introduce non-linearities into the network. Commonly used activation functions like ReLU (Rectified Linear Unit), Leaky ReLU, or Exponential Linear Unit (ELU) can be employed. ReLU, in particular, is preferred for its simplicity and effectiveness in handling non-linear transformations. We also looked at SELU as an option.

The output layer consists of a single neuron, representing the predicted maximum enclosed area. Like the input and hidden layers, this layer is implemented as a linear layer.

For training the neural network, Mean Squared Error (MSE) is typically chosen as the loss function for regression problems like this one. MSE calculates the

average squared difference between the predicted values and the actual values, providing a measure of the model's performance. We also implemented Mean Absolute Percentage Error (MAPE) as well to gauge the performance.

Moreover, for the optimiser, Stochastic Gradient Descent (SGD) and Adaptive Moment Estimation (ADAM) are the two main options that we will consider following our research.

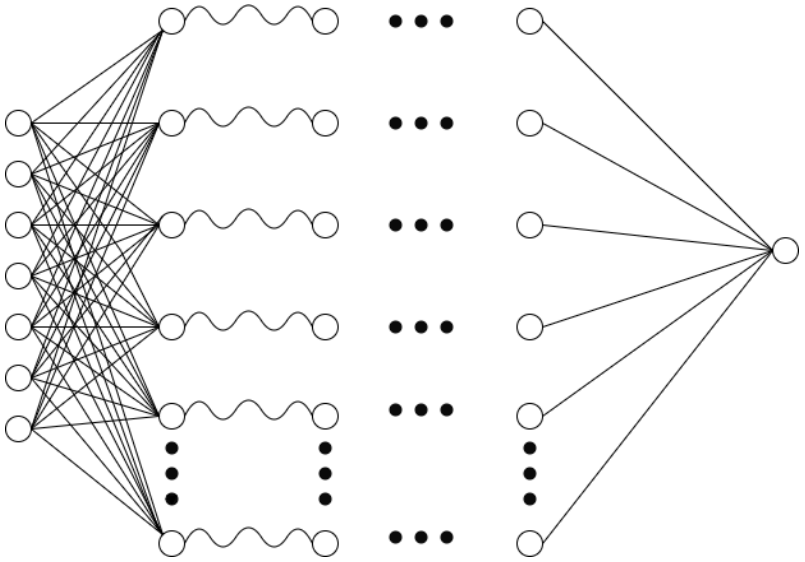


Figure 2 - Simple Neural Network Suited for this Problem

Pt2. Python Implementation

Firstly, we present the code prior to disambiguation for context. This involves the importing of the necessary libraries, the loading of the data, and the making of the data to be dimensionally-homogeneous, scaled, as well as the altering of its dimensionality by computing the cartesian product. Here are the following code snippets, which we consider to be self-explanatory and will thus not go into detail regarding them.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import itertools
from sklearn.preprocessing import RobustScaler
from sklearn.preprocessing import StandardScaler
import os
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm
from math import floor
from sklearn.model_selection import train_test_split
from torch.optim.lr_scheduler import StepLR
import copy
```

```
# Loading of three provided datasets
x_train = pd.read_csv("/content/x_train_id.csv")
y_train = pd.read_csv("/content/y_train_id.csv")

# Selecting of the columns containing the inputs
transf_cols = [col for col in x_train.columns if col != 'id']
transf_cols
```

```
# Function to normalise the inputs according to their corresponding sum (perimeter)
def perimeter_normalize_inputs(data):
    # Calculate the perimeter for each row using only the specified columns
    perimeters = data[transf_cols].sum(axis=1)
    # Create a copy of x_train to avoid modifying the original DataFrame
    x_train_normalized = data.copy()
    # Normalize specified columns by dividing each side length by the total perimeter
    x_train_normalized[transf_cols] = data[transf_cols].div(perimeters, axis=0)

    return x_train_normalized, perimeters.tolist()

# Function to normalise the outputs according to the sum of the inputs squared (perimeter squared)
def perimeter_normalize_outputs(perimeters, data):
    # Iterate over each row index and its corresponding perimeter
    for index, perimeter in enumerate(perimeters):
        # Divide the 'expected' value by the corresponding perimeter squared
        data.at[index, 'Expected'] /= (perimeter ** 2)
    return data

# Function to make the transform the outputs back to their expected form.
def reverse_perimeter_normalize_outputs(perimeters, data):
    # Iterate over each row index and its corresponding perimeter
    for index, perimeter in enumerate(perimeters):
        # Multiply the 'Expected' value by the square of the corresponding perimeter
        data.at[index, 'Expected'] *= (perimeter ** 2)
    return data
```

```
# Making of copies of the dataframes to ensure that the original dataframes are not altered
x_data = x_train.copy()
y_data = y_train.copy()

# Scaling and homogenisation of inputs and outputs using above methods
scaled_homogenized_inputs, train_perimeters= perimeter_normalize_inputs(x_data)
scaled_homogenized_outputs = perimeter_normalize_outputs(train_perimeters, y_data)
scaled_homogenized_inputs = scaled_homogenized_inputs.drop(columns=['id'])
```

```
# Function to compute the cartesian product of the inputs, increasing the dimensionality
def compute_multiplications(x_train):
    # Drop the 'id' column if it exists
    if 'id' in x_train.columns:
        x_train = x_train.drop(columns=['id'])

    # Get all combinations of 2 from column names
    combinations = list(itertools.combinations_with_replacement(x_train.columns, 2))

    # Initialize an empty list to store results
    rows_result = []

    # Compute multiplication for every row and each combination
    for _, row in x_train.iterrows():
        row_result = {}  # Dictionary to store multiplication results for this row
        for comb in combinations:
            col_name = f"{comb[0]}_{comb[1]}"  # New column name

            # Ensure that the combination is in the correct order
            if comb[0] <= comb[1]:
                row_result[col_name] = row[comb[0]] * row[comb[1]]
        rows_result.append(row_result)

    # Create DataFrame from the list of row results
    result_df = pd.DataFrame(rows_result)

    return result_df

# Calling of the above function
cartesian_inputs = compute_multiplications(scaled_homogenized_inputs)
```

```
# Get input columns
cart_transf_cols = [col for col in cartesian_inputs.columns if col != 'id']

# Get outputs in list form
outputs = scaled_homogenized_outputs['Expected'].tolist()

# Reshape to outputs to numpy array of (size, 1) instead of (size,), expected input for pytorch
np_outputs = np.array(outputs)
y_train_reshaped = np_outputs.reshape(-1, 1)

# Compile all inputs into a list of lists of integers
inputs = cartesian_inputs[cart_transf_cols].values.tolist()
```

D. Disambiguating the Symmetries and Generating Training Data

```
# Sort each inner list representing one input
inputs = [sorted(inner_list) for inner_list in inputs]

# Function to remove any duplicate inputs post sorting
def remove_duplicates(inputs):
    unique_tuples = set()
    result = []
    for inner_list in inputs:
        inner_tuple = tuple(inner_list)
        if inner_tuple not in unique_tuples:
            result.append(inner_list)
            unique_tuples.add(inner_tuple)
    return result

# Calling of function remove duplicate inputs
disambiguated_inputs = remove_duplicates(inputs)
```

In the above image the code disambiguates the data to remove any potential duplicates in the data based on the symmetries discussed earlier in the paper (translation, reflection, rotation, permutations of edges). Initially we compile all the data of the inputs into a list of a list of inputs, each list representing the 28 inputs. We then go through each list and compare those lists to other already checked lists to see if any two lists of 28 edges have the exact same set of edges. This is done through the comparison of tuples as there are very simple provided methods for checking if two tuples have the same set of elements as for lists. First each list is turned into a tuple and if it does not already exist in the set of unique and checked tuples we add it to that set, otherwise we don't. We are then left with a set of all the unique sets of 28 edges. The sorting also ensures that all inputs are in one, desired form, disambiguating the data.

```
# Function to double check for potential input duplicates (since no duplicates were found)
def get_duplicate_indices(df, column_name):
    duplicate_indices = []
    seen_values = set()
    for index, row in df.iterrows():
        value = row[column_name]
        if value in seen_values:
            duplicate_indices.append([df[df[column_name] == value].index[0], index])
        else:
            seen_values.add(value)
    return duplicate_indices

# Calling of function
indices_of_duplicates = get_duplicate_indices(y_train, 'Expected')
```

The above code was created following the fact that we found no symmetrical inputs. We used it to double check that any existing inputs with the same maximal areas were indeed not permutations of one another. The function *get_duplicate_indices* returns all pairs of indices that correspond to inputs with the same maximal areas, and we used its results to manually check whether these inputs were indeed not permutations of one another, and missed by our *remove_duplicates* function above. They were not.

E. Neural Network Training:

.

```
import os
import torch
from torch import nn
from torch.utils.data import DataLoader, Dataset
from tqdm import tqdm
from math import floor
from sklearn.model_selection import train_test_split
from torch.optim.lr_scheduler import StepLR
```

```
datatype=torch.float32

# Split data into train and validation data respectively
x_train_data, x_test_data, y_train_data, y_test_data = train_test_split(scaled_disambiguated_inputs,
            scaled_outputs, test_size=0.2, random_state=42)

# Ensure that the elements are lists of float32
x_train_data = [list(map(np.float32, sublist)) for sublist in x_train_data]
x_test_data = [list(map(np.float32, sublist)) for sublist in x_test_data]
y_train_data = [list(map(np.float32, sublist)) for sublist in y_train_data]
y_test_data = [list(map(np.float32, sublist)) for sublist in y_test_data]
```

Firstly, we split the data into training and validation data, ensuring that the elements are all of type float32. The training data will be used to optimise the model, whilst the validation data will be used to evaluate it.

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28, 256, bias=False),
            nn.SELU(0.1),
            nn.Linear(256, 128, bias=False),
            nn.SELU(0.1),
            nn.Linear(128, 64, bias=False),
            nn.SELU(0.1),
            nn.Linear(64, 32, bias=False),
            nn.SELU(0.1),
            nn.Linear(32, 1, bias=False),
        )

    def forward(self, x):
        logits = self.linear_relu_stack(x)
        return logits
```

Next, we have the final architecture of the neural network, starting with the input layer consisting of 28 neurons, and then going into the biggest hidden layer of 256 neurons. The SELU activation function was used after in-depth tests, as it was the one which led to the greatest accuracy. The network consists of five hidden layers, with the number of neurons being halved in each subsequent layer until the final layer, which jumps from 32 to the output layer with one neuron. We decided to have the neurons in the hidden layers be halved with each layer as this provided us with greater performance than keeping the number of neurons constant across all of the layers.

```
class NumpyDataset(Dataset):
    def __init__(self, x, y, dtype=datatype):
        self.x = torch.tensor(x, dtype=dtype)
        self.y = torch.tensor(y, dtype=dtype)

    def __repr__(self):
        return f"NumpyDataset(data={self.x, self.y})"

    def __len__(self):
        return self.x.size()[0]

    def __getitem__(self, index):
        return self.x[index], self.y[index]
```

Next, we used the *NumpyDataset* class to integrate NumPy arrays with PyTorch's data handling system. The *__init__* method converts input features (*x*) and target labels (*y*) from NumPy arrays into PyTorch tensors of a specified data type (*dtype*). The *__repr__* method provides a string representation of the dataset, showing the data tensors. The *__len__* method returns the number of samples in the dataset, and the *__getitem__* method retrieves individual samples and their labels by index. This class simplifies the use of NumPy data in PyTorch, making it easier to handle data during model training.

```
#Method converting tabel data to tensors
def np_to_tensor(x, dtype=datatype):
    return torch.tensor(x, dtype=dtype)

#Uses above method to set up tensor data
def prepare_data(x_train, y_train, x_test, y_test):
    data_dict = {}
    data_dict["train"] = NumpyDataset(x_train, y_train)
    data_dict["x_test"] = np_to_tensor(x_test)
    data_dict["y_test"] = np_to_tensor(y_test)
    return data_dict

# Callling of the functions
training_data = prepare_data(x_train_data, y_train_data, x_test_data, y_test_data)
```

Then, we used the above code to convert table data into PyTorch tensors and prepare the data for model training and testing. The *np_to_tensor* function converts NumPy arrays (*x*) to PyTorch tensors using a specified data type *dtype*, while the *prepare_data* function uses this conversion method to set up training and testing data. It creates a dictionary *data_dict* where the training data is stored as a *NumpyDataset* instance, and the testing data is converted to tensors using the *np_to_tensor* function. Subsequently, the data is in the appropriate form for PyTorch operations and model training

```
# Check if CUDA is available
if torch.cuda.is_available():
    devicet = torch.device("cuda")
    print("CUDA is available. Using GPU.")
else:
    devicet = torch.device("cpu")
    print("CUDA is not available. Using CPU.")

# Print device name
print(f"Using device: {devicet}")
```

Next, we check whether cuda is operational to ensure that we are running the model on the GPU, at increased speed.

```
def train_model(
    data, model, nepochs=1, lr=1e-6, batch_size=1, print_every=1, loss_fn=nn.MSELoss()
):
    torch.manual_seed(0)

    train_data = data["train"]
    x_test = data["x_test"]
    y_test = data["y_test"]

    # Move the model to the GPU
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = model.to(device)
    loss_fn = loss_fn.to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    loss_dict = {"train": [], "test": []}
    num_batches = floor(len(train_data) / batch_size)

    # Initialize variables to keep track of the best model and the lowest test loss
    best_model = None
    lowest_test_loss = float('inf')
```

This is the beginning part of the function actually training the model. The device on which the model will be run is set up, the loss function set up, the optimiser set up, and the number of batches calculated. Subsequently, the variables to keep track of the best model and lowest test loss are set up.

```
for epoch in tqdm(range(nepochs)):

    epoch_loss_sum = 0

    for x_batch, y_batch in DataLoader(
        train_data, batch_size=batch_size, shuffle=True
    ):
        # Move batch to the GPU
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)

        y_pred = model(x_batch)
        loss = loss_fn(y_pred, y_batch)
        epoch_loss_sum += loss.item()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    loss_dict["train"].append(epoch_loss_sum / num_batches)

    with torch.no_grad():
        # Move test data to the GPU
        x_test, y_test = x_test.to(device), y_test.to(device)

        y_pred = model(x_test)
        loss = loss_fn(y_pred, y_test)
        loss_dict["test"].append(loss.item())

        # Check if this is the lowest test loss we've encountered
        if loss.item() < lowest_test_loss:
            lowest_test_loss = loss.item()
            # Make a deep copy of the model to store the best model
            best_model = copy.deepcopy(model)
            print(loss.item())

        if (epoch + 1) % print_every == 0:
            print(
                #add to print best model as well
                epoch + 1, f"Train: {loss_dict['train'][-1]}  Test: {loss.item()}"
            )

    # Ask the user whether to continue every 5000 epochs
    if (epoch + 1) % 5000 == 0:
        user_input = input("Do you want to continue training? (y/n): ")
        if user_input.lower() != 'y':
            print("Training stopped by user.")
            break
```

The above code shows what goes on within each epoch. As expected, the model parameters are updated based on training data batches and the model performance is evaluated on the test data periodically. The *DataLoader* class is used to handle the batching and shuffling of training data. Within each epoch,

the loop calculates the loss for each batch of training data, updates the model parameters using backpropagation, and records the average loss for the epoch. Additionally, it evaluates the model's performance on the test data, keeping track of the lowest test loss encountered and storing the best model accordingly. Every 5000 epochs, the user is prompted to decide whether to continue training. The progress of training is displayed periodically, printing the current epoch number, the training loss, and the test loss.

```
# Move model and data back to CPU
best_model = best_model.to("cpu")
model = model.to("cpu")
data["x_test"] = x_test.to("cpu")
data["y_test"] = y_test.to("cpu")

# Create a new list for train_data with all tensors moved to the CPU
new_train_data = [(x.to("cpu"), y.to("cpu")) for x, y in train_data]
data["train"] = new_train_data

return model, best_model, loss_dict
```

The last part of the *train_model* method is responsible for moving the model and data tensors back to the CPU after training. It ensures that the tensors are in a format accessible by the CPU for further processing or inference. First, it moves both the *model* and *best_model* to the CPU using the *.to("cpu")* method. Then, it converts the test data tensors (*x_test* and *y_test*) back to CPU tensors. Lastly, it creates a new list called *new_train_data*, where each batch of training data is converted to CPU tensors. This new list is then assigned to the data dictionary under the key "*train*". Finally, the function returns the updated *model*, *best_model*, and *loss_dict*. This step ensures that all tensors are in a consistent format suitable for further analysis or evaluation.

```python
class MAPELoss(torch.nn.Module):
    def __init__(self):
        super(MAPELoss, self).__init__()

    def forward(self, y_pred, y_true):
        """
        Calculates the Mean Absolute Percentage Error between y_pred and y_true.

        Args:
            y_pred (torch.Tensor): Predicted values.
            y_true (torch.Tensor): True values.

        Returns:
            torch.Tensor: Mean Absolute Percentage Error.
        """
        epsilon = 1e-8  # to avoid division by zero
        percentage_error = torch.abs((y_true - y_pred) / (y_true + epsilon))
        mean_percentage_error = torch.mean(percentage_error)*100
        return mean_percentage_error
```

Next, the *MAPELoss* class defines a custom loss function for PyTorch neural networks, specifically computing the Mean Absolute Percentage Error (MAPE) between predicted (*y_pred*) and true (*y_true*) values. In the forward method, it calculates the absolute percentage error for each prediction by taking the absolute difference between true and predicted values, divided by true values plus a small epsilon value (1e-8) to prevent division by zero. It then computes the mean of these absolute percentage errors across all predictions and multiplies it by 100 to express the error as a percentage. The resulting mean absolute percentage error is returned as the output of the forward method, used as a loss metric during model training to minimise the MAPE.

```python
# Define model arguments
total_epochs = 50000
print_every = 100
batch_size = 56
lr = 0.00001
loss_fn = MAPELoss()
```

```python
model = NeuralNetwork()

model, best_model, loss_dict = train_model(
    training_data,
    model,
    nepochs=total_epochs,
    batch_size=batch_size,
    lr=lr,
    print_every=print_every,
    loss_fn=loss_fn,
)
```

Finally, the model hyperparameters are set up, and the model ran with the appropriate values. An example of the output of this model when run is shown below:

```
| 1/100000 [00:00<9:23:36,  2.96it/s]25.00054931640625
| 2/100000 [00:00<9:04:41,  3.06it/s]12.73844051361084
| 3/100000 [00:00<8:49:10,  3.15it/s]4.37299919128418
| 4/100000 [00:01<9:11:21,  3.02it/s]2.9947097301483154
| 5/100000 [00:01<9:01:28,  3.08it/s]2.153289318084717
| 6/100000 [00:01<9:02:37,  3.07it/s]1.6239166259765625
| 7/100000 [00:02<9:01:46,  3.08it/s]1.2590397596359253
| 8/100000 [00:02<8:55:47,  3.11it/s]1.0525104999542236
| 9/100000 [00:02<8:50:50,  3.14it/s]0.8971519470214844
| 10/100000 [00:03<8:52:48,  3.13it/s]0.81121426820755
| 11/100000 [00:03<8:51:10,  3.14it/s]0.7174814939498901
| 12/100000 [00:03<8:49:55,  3.14it/s]0.6477273106575012
| 13/100000 [00:04<8:42:51,  3.19it/s]0.5976862907409668
| 14/100000 [00:04<8:47:51,  3.16it/s]0.5435762405395508
| 15/100000 [00:04<8:44:11,  3.18it/s]0.509289026260376
| 17/100000 [00:05<8:41:56,  3.19it/s]0.4741379916667938
| 18/100000 [00:05<8:42:35,  3.19it/s]0.43626031279563904
| 19/100000 [00:06<8:40:24,  3.20it/s]0.4302242398262024
| 20/100000 [00:06<8:41:53,  3.19it/s]0.39325034618377686
| 22/100000 [00:07<9:19:17,  2.98it/s]0.3686661422252655
| 23/100000 [00:07<9:40:47,  2.87it/s]0.3566409945487976
| 28/100000 [00:09<11:08:57,  2.49it/s]0.31171873211860657
| 32/100000 [00:10<9:10:16,  3.03it/s]0.3029778003692627
| 34/100000 [00:11<8:46:46,  3.16it/s]0.28239428997039795
```

Firstly, all of the lowest test MAPE values are printed for reference of the best current model. The corresponding epoch of each of these values is also provided.

```
| 100/100000 [00:32<8:25:45,  3.29it/s]100 Train: 0.1844869051915659  Test: 0.19819466769695282
```

Moreover, at every 100 epochs, the current training and validation MAPEs are printed.

```
  5%|▌         | 4999/100000 [28:32<7:50:43,  3.36it/s]5000 Train: 0.05358526077856061  Test: 0.06238071992993355
Do you want to continue training? (y/n): y
```

Lastly, after every 5000 epochs, the model halts temporarily and expects an input to know whether or not to continue.

```
def plot_loss(loss_dict):
    plt.plot(loss_dict["train"], label="train")
    plt.plot(loss_dict["test"], label="test")
    plt.yscale("log")
    plt.legend()
    plt.show()
```

```
plot_loss(loss_dict)
```

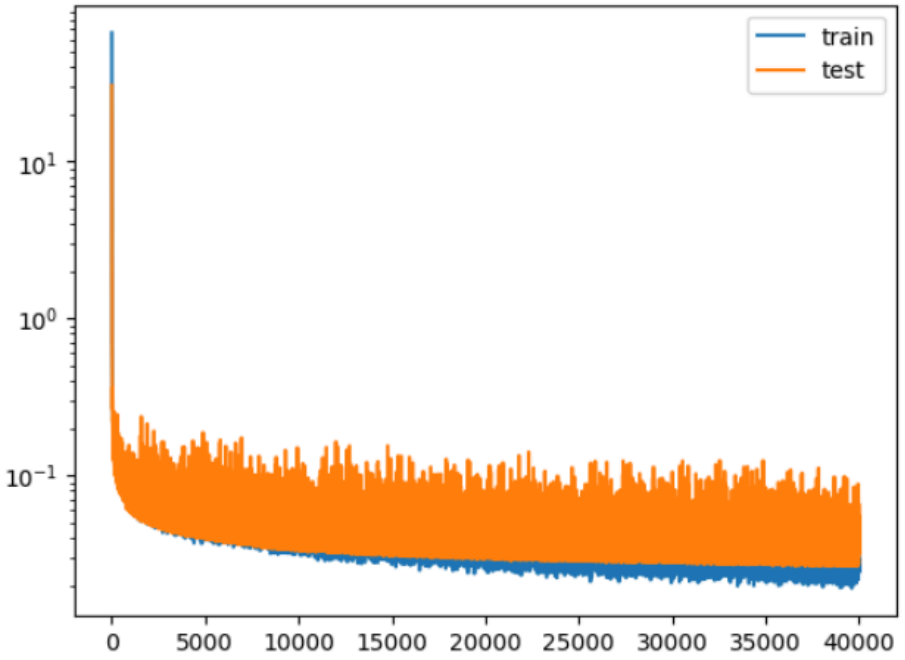


Figure 3 - Training vs Test Loss of Model

Once the model finishes training, we create diagrams to show the training and testing losses (above), as well as to show the performances of the final versus best models on the data (below).

```python
def plot_result(data_dict, model):
    with torch.no_grad():
        y_pred = model(data_dict["x_test"])
        loss = loss_fn(y_pred, data_dict["y_test"])
        print(loss.item())
        plt.plot(y_pred.cpu().numpy(), data_dict["y_test"].cpu().numpy(), "o")
        # plot line x = y
        x = [0.02, 0.09]
        plt.plot(x, x, "r")
        plt.show()

plot_result(training_data, model)
plot_result(training_data, best_model)
```

```
0.03184179216623306
```

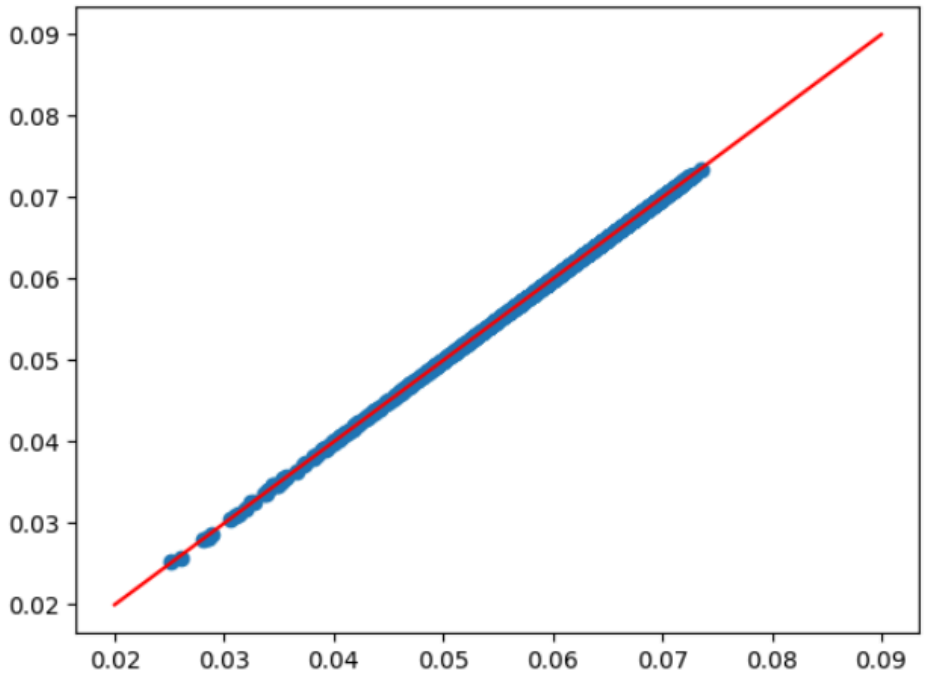


Figure 4 - Performance of Final Model

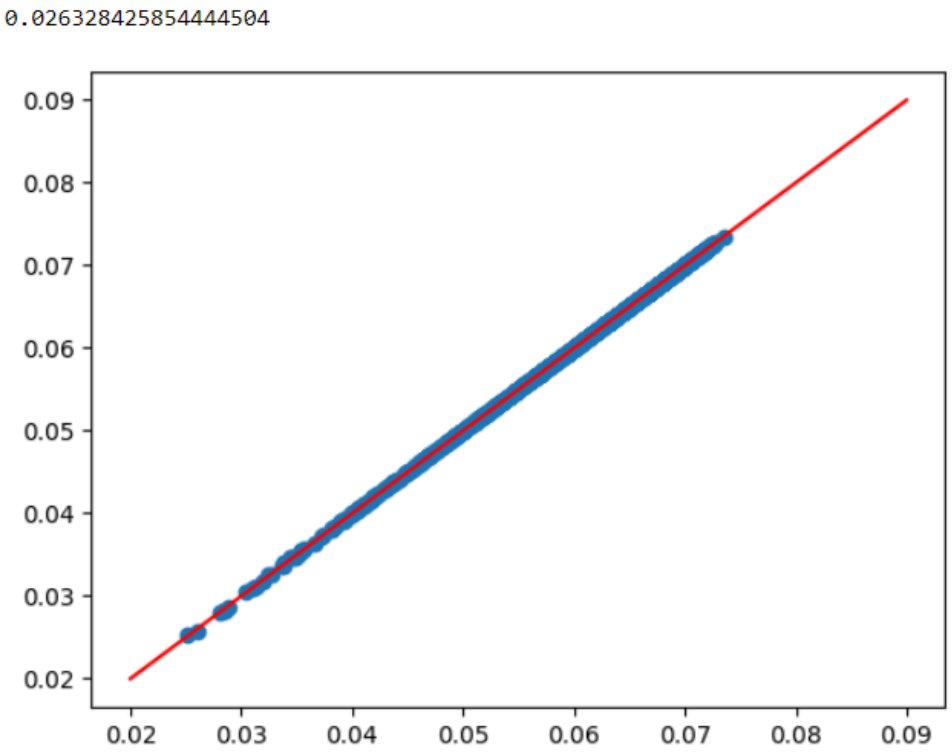


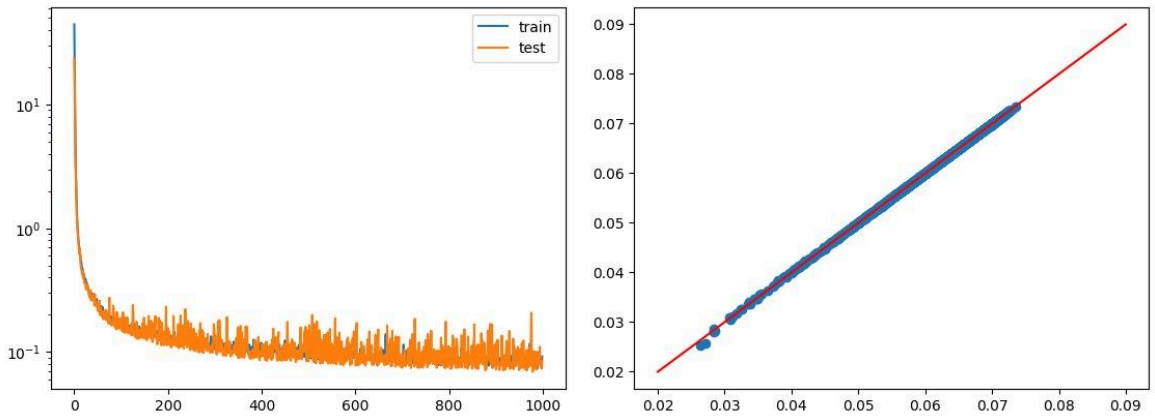
Figure 5 - Performance of Best Model

F. Network Comparison - With vs Without Respecting Invariances:
With respecting invariances:

Before training the model, we ensured that the input data respected invariance by sorting each inner list of inputs and removing any duplicates. Respecting invariance in this context means maintaining the properties of the heptagon's side lengths such that the model's performance and predictions are not influenced by the order of the sides or redundant data points. By sorting the side lengths, we made sure that the model treats different permutations of the same set of side lengths equivalently, thereby focusing on the actual geometrical properties rather than their ordering. Removing duplicates further streamlined the data, ensuring that each unique set of side lengths is represented only once, which helps in reducing redundancy and improving the efficiency and accuracy of the model. This preprocessing step is crucial, and demonstrated below by the accuracy of a model trained with and without considering it. We trained it for 1000 epochs with the exact same model architecture for both runs and the results are shown below.

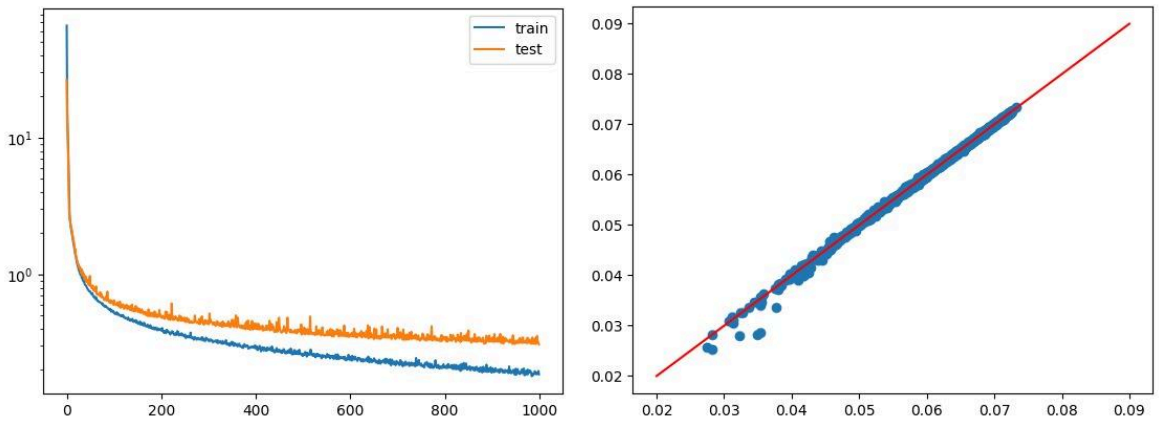
With Respecting Invariance:

Final MAPE: 0.0843172



Without Respecting Invariances:

Final MAPE: 0.30806618



As demonstrated above the model is almost four times more accurate when invariance is respected. Furthermore, when training for more epochs the loss plateaus much later and at a smaller MAPE when invariance is respected, showing how vital a component it is when attempting to create a model which is as accurate as possible.

Pt3. Universal Approximation Results:

G. Qualitative Report on Universal Approximation Theorem (UAT):

The universal approximation theorem states that every continuous function on a given domain can be approximated to any required degree of accuracy by a feedforward neural network (FNN) with at least one hidden layer. A shallow network refers to a neural network with a relatively small number of

hidden layers, perhaps even a single one, whilst a deep network has a relatively great number of hidden layers. Since the UAT can be applied to both types of networks, the theorem conveys that both simple and complex neural networks have the ability to generate functions that comprehend and predict patterns in any dataset.

This theorem is of great importance since it highlights the computational prowess and consequent potential that FNNs possess in garnering deep understandings of real world problems. The fact that any continuous function can be approximated to the desired degree of accuracy by an FNN, combined with the reality that a great number of real world problems can be represented by continuous functions, underscores the ability of FNNs to create significant, real-world impacts.

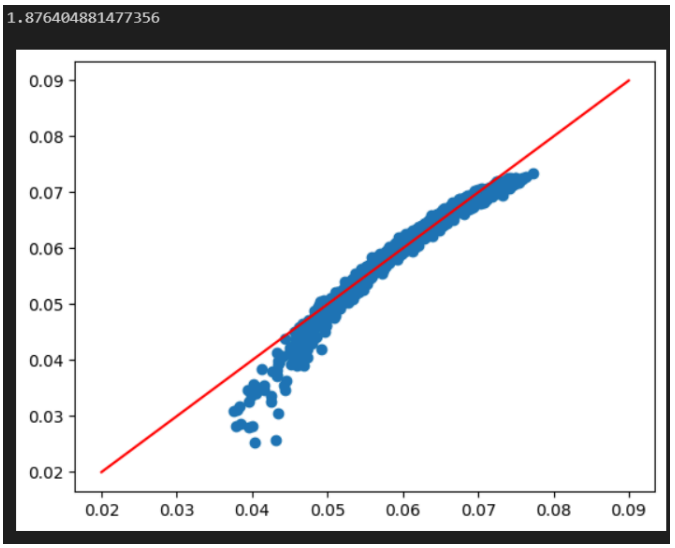
Nevertheless, this is slightly more complicated than implied by the theorem and depends on a number of factors. For instance, shallow networks will, in some cases, struggle to produce the same results as deeper networks if their hyperparameters are identical. Moreover, the actual performance of a model, and its ability to ultimately produce the desired degree of accuracy for a given function, will depend on the optimisation of a number of these hyperparameters, such as the learning rate, batch size, number of epochs, activation functions, optimiser types, and the number of neurons in a given layer. Thus, though the model correctly states that FNNs can be used to attain a given desired accuracy for a provided function, this is not necessarily a straightforward process, and expertise of the different hyperparameters relevant to FNNs is often essential.

All in all, the universal approximation theorem underscores the potential real-world impact of FNNs without detailing the actual processes that underlie its potential success. Though this does not take away from the truth of its message, empirical evidence must still be collected in order to understand the actual mechanisms that make the universal approximation theorem hold, and also to check its validity. This will be carried out in the next section.

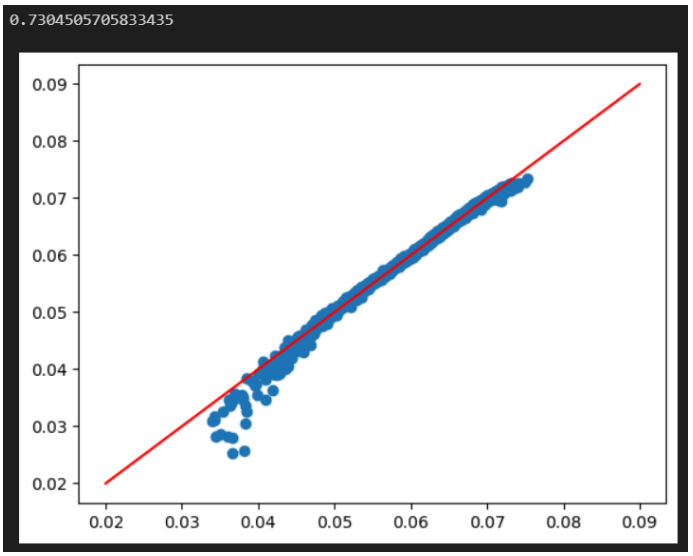
H. Empirical Check of the Validity of Universal Approximation Theorem:
These training parameters used for each test remained the same:

```
# Define model arguments
total_epochs = 100
print_every = 10
batch_size = 56
lr = 0.00001
loss_fn = MAPELoss()
```

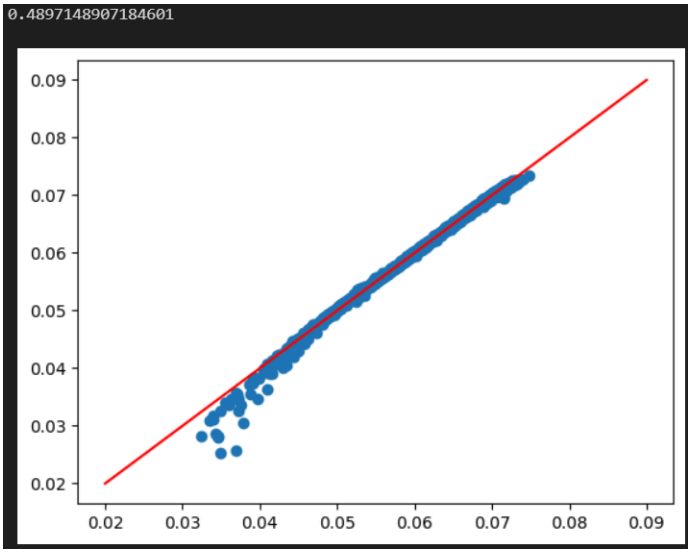
1st test: 1 hidden layer 50 neurons



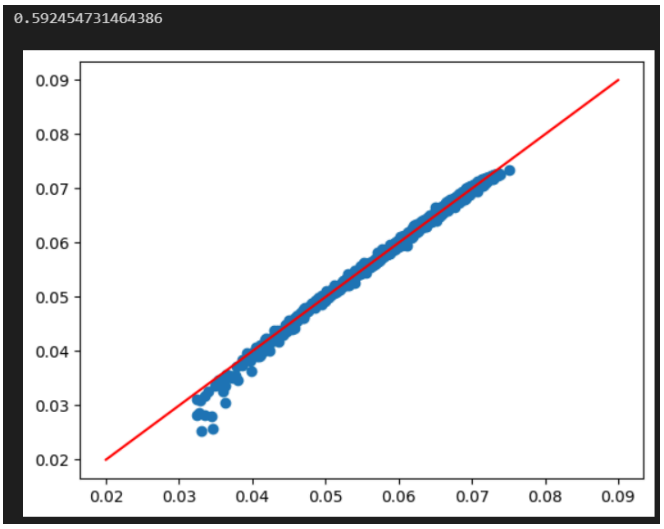
2nd test: 1 hidden layer 200 neurons



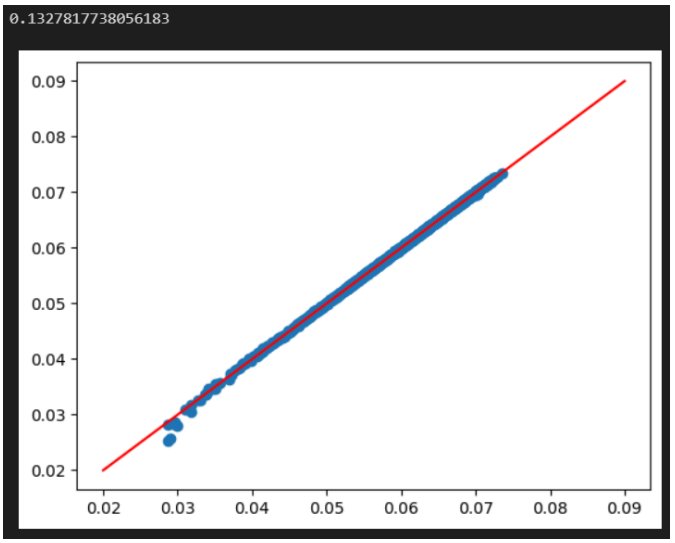
3rd test: 1 hidden layer 400 neurons



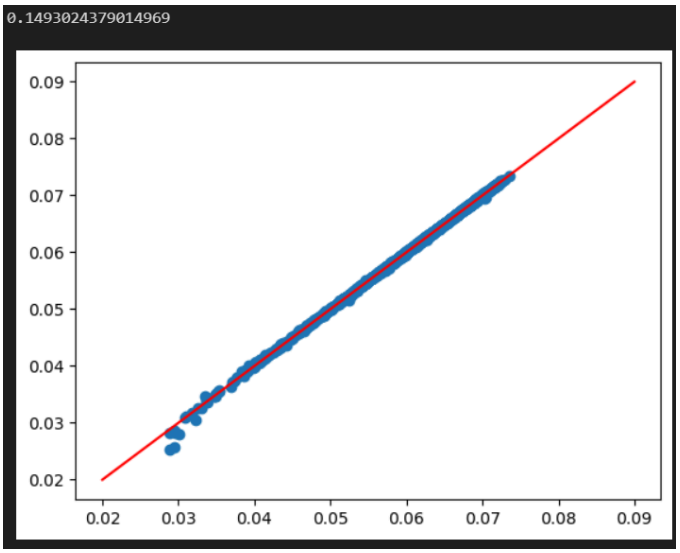
4th Test: 3 hidden layers 50 neurons



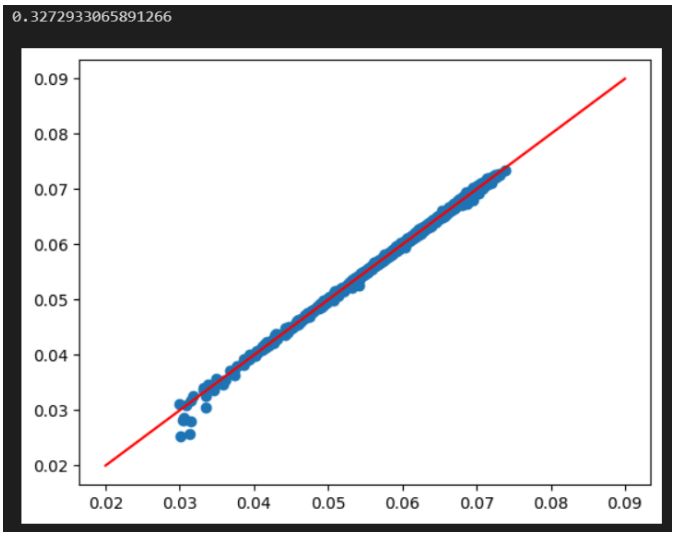
5th Test: 3 hidden layers 200 neurons



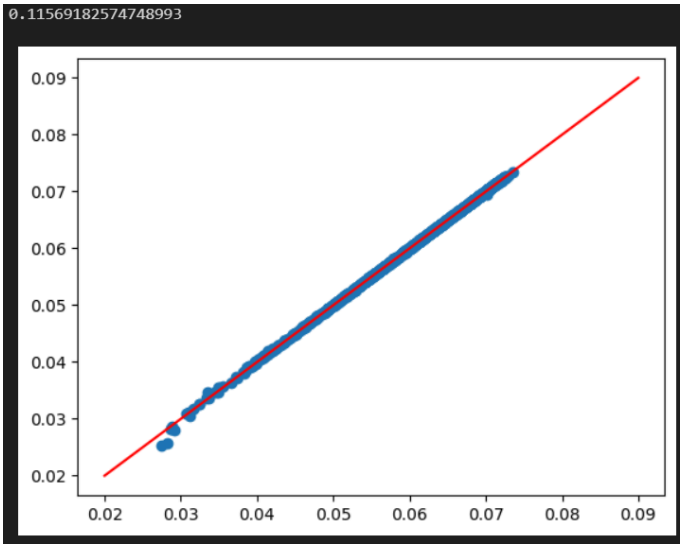
6th Test: 3 hidden layers 400 neurons



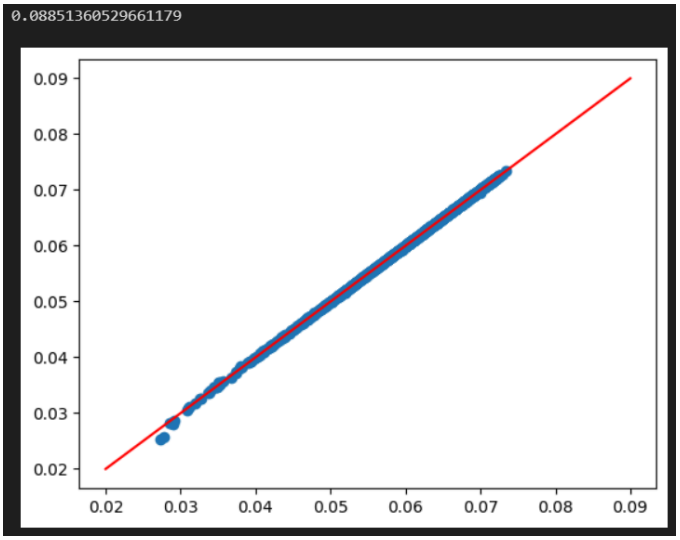
7th Test: 5 hidden layers 50 neurons



8th Test: 5 hidden layers 200 neurons

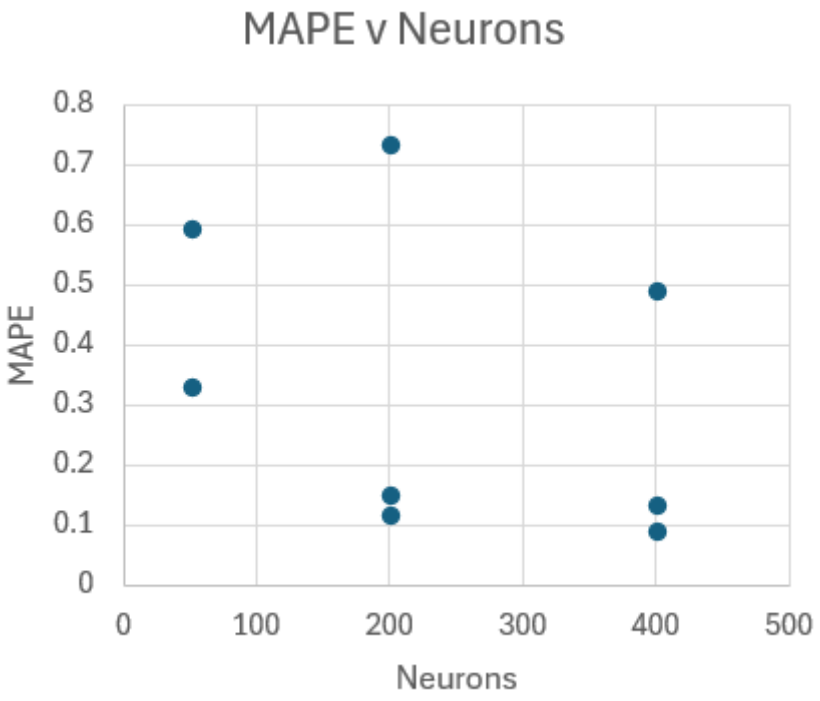


9th Test: 5 hidden layers 400 neurons



All the graphs above plot the comparison of the predicted values of a model to the true values from a test dataset, and at the top shows their MAPE. Below we also present the data in a table 2 graphs showing the relationship between MAPE and the hidden layers and neurons respectively. Note both graphs ignore the very first line of the data below as it is too large to be properly represented in the graph relative to the other values.

hidden layers	neurons	MAPE
1	50	18.764
1	200	0.7304
1	400	0.4897
3	50	0.5925
3	200	0.1493
3	400	0.1328
5	50	0.3273
5	200	0.1157
5	400	0.0885



graph of MAPE against Neurons per layer, highest points to lowest are 1,3,5 respectively. First line of data is excluded

Careful examination of the data and results reveals some clear patterns. Increasing the number of neurons per hidden layer and the number of hidden

layers reduces the error, leading to a better fit. This makes sense logically: more trainable parameters provide greater flexibility for the model to adjust and fit the curve to the data more accurately. Notably, the performance difference is substantial when the Mean Absolute Percentage Error (MAPE) is greater than 0.3 but diminishes below 0.3. This occurs because models start to plateau in training as the error decreases, making small changes have a larger impact on progress. However, even in these cases, models with more parameters consistently reach better results more quickly.

Examining the graphs further, the influence of more neurons per hidden layer and more hidden layers becomes evident. The number of layers seems to have a more significant effect on the fit than the number of neurons. With a single layer, the data points appear more curved, and increasing the neurons helps fit the line to these curves better. In subsequent graphs with more layers, the data points align more linearly, showing a stronger impact of the additional layers. Although increasing neurons produces more accurate models even when the fit is close to the line, the improvement from 1 to 3 hidden layers is more pronounced than from 3 to 5 hidden layers.

The observed patterns underscore the principles of the universal approximation theorem, which states that a neural network with at least one hidden layer can approximate any continuous function, given sufficient neurons. However, in practice, adding more layers (depth) and neurons (width) enhances the model's capacity to approximate complex functions more accurately. While diminishing returns are evident as the error becomes smaller, the overall trend supports the notion that deeper and wider networks achieve better performance, aligning with theoretical expectations.